# COMP2511

Tutorial 7

# Last Tutorial

- Strategy Pattern
- Observer Pattern
- Composite Pattern
- Factory Pattern

# This Tutorial

- Decorator Pattern
- Singleton Pattern
- Abstract Factory Pattern

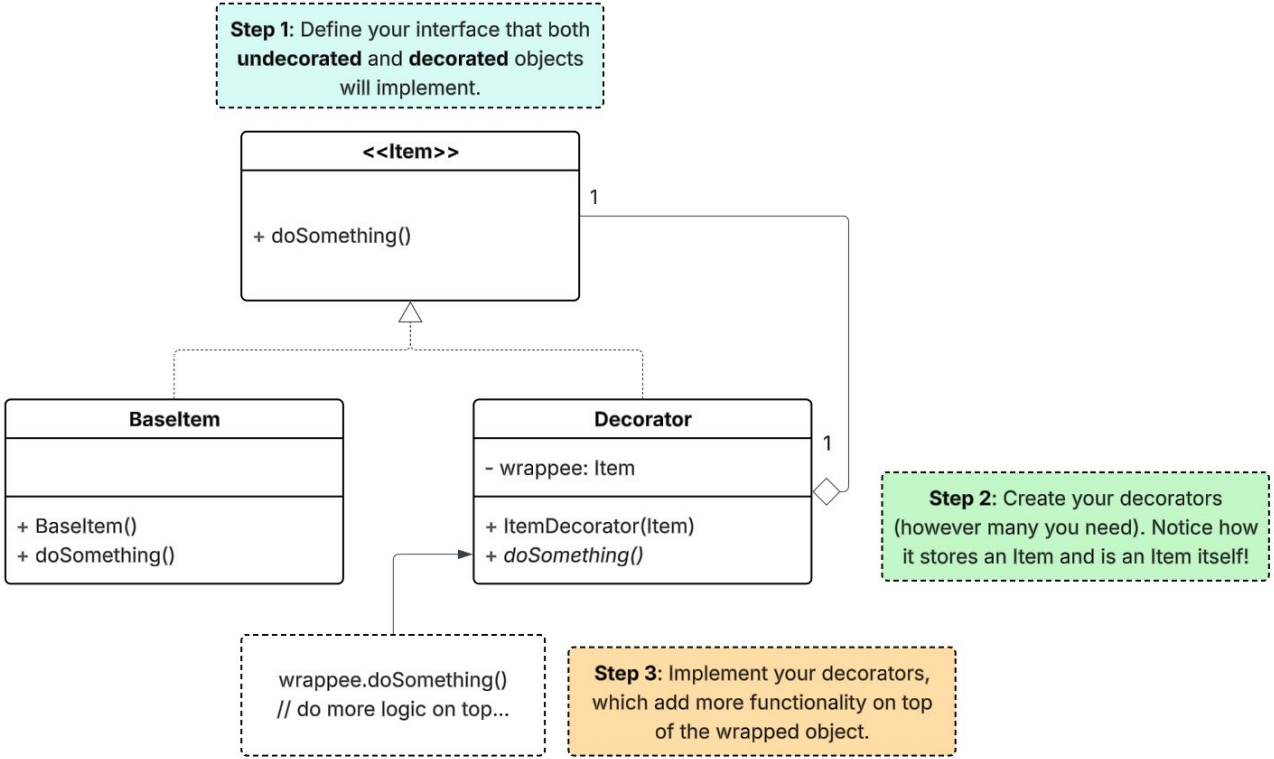
# Decorator Pattern

- Suppose we have vanilla, chocolate and strawberry ice cream and we were modelling different combinations of scoops as classes. For example,
  - `VanillaIceCream`
  - `VanillaAndStrawberryIceCream`
  - `ChocolateAndStrawberryIceCream`
  - `VanillaChocolateAndStrawberryIceCream`
  - and so on...
- What is the issue with this approach?
  - We would end up with *way* too many classes.
- The Decorator Pattern will help model different combinations of functionalities that can be added on top of each other.

# Decorator Pattern

- The Decorator Pattern is a design pattern that allows new behaviours to be attached on top of an existing object.
- The existing object gets stored inside another object, referred to as a **decorator**.
- The decorator uses the original functionality of the wrapped object, then modifies it or adds additional functionality.
    - Both the unwrapped and wrapped objects need to implement a common interface. This allows us to store decorated objects within decorators, so they can be stacked.
    - In the ice cream example, the unwrapped object would be the ice cream cone and the decorators would be different scoops of ice cream. We are adding more scoops on top of the ice cream cone, and they can be added in any order.

# Decorator Pattern Implementation



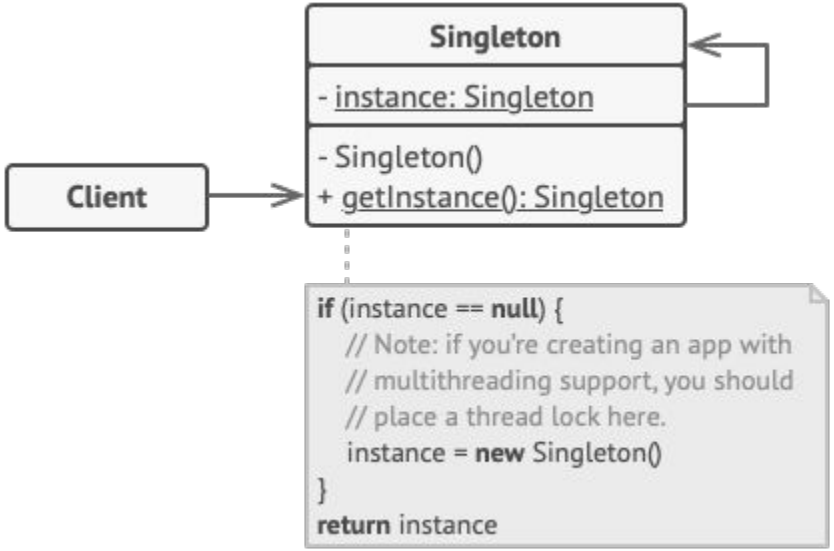
# Live Example: Decorator Pattern

- Develop a system in **src/icecream** to model ice cream orders. An ice cream cone costs $0.70 and customers are able to order any combination of the following flavours:
  - Vanilla, costing $4.00 per scoop.
  - Strawberry, costing $4.50 per scoop.
  - Chocolate, costing $5.00 per scoop.

# Singleton Pattern

- The Singleton Pattern is a design pattern that ensures that only one instance of a class exists at any point in time.
  - This may be desired when you need to have one “global instance” of a class.
- The implementation is quite simple:
  - Make the class constructor private.
  - Make the class store a static instance of itself (this instance is the **singleton**).
  - Write a static getter that returns the instance if it exists or constructs it if it doesn’t.
- Lab 7 involves solving a concurrency issue. In Java, the **`synchronized`** keyword can be added to methods to prevent an object or class from being concurrently accessed by multiple threads.

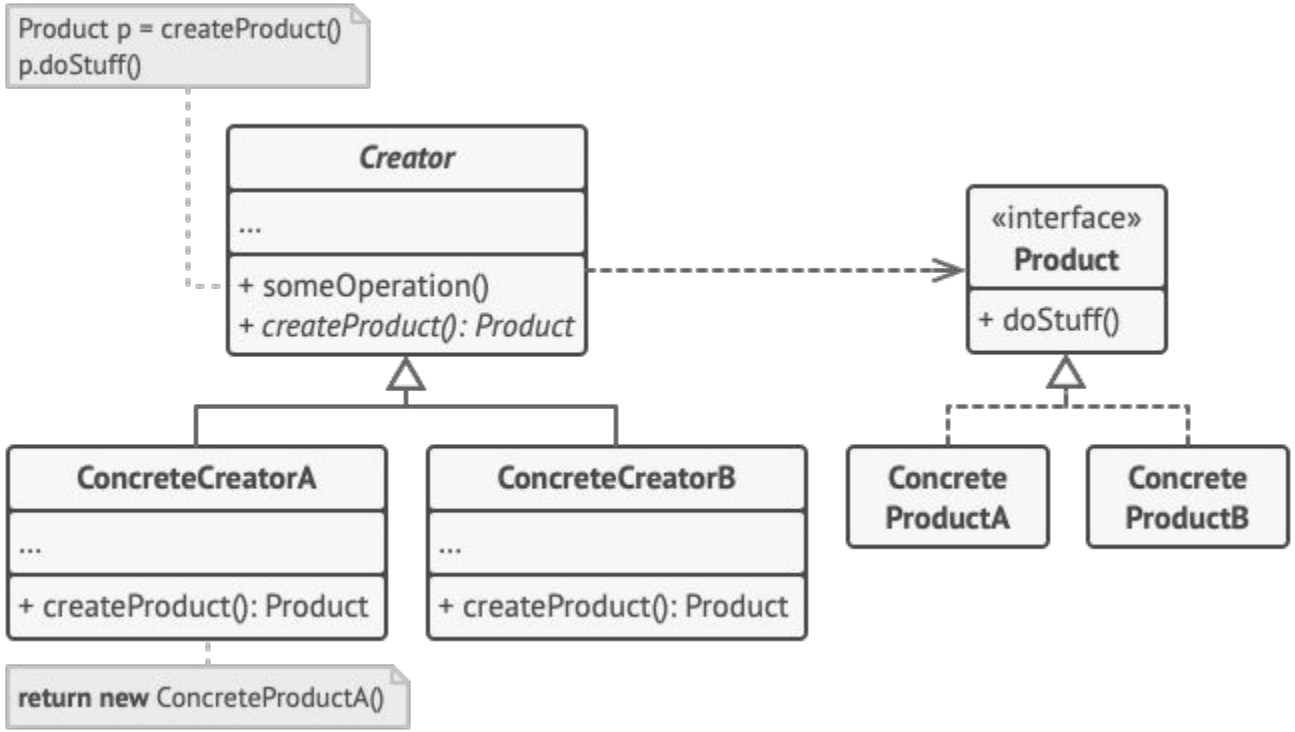
# Singleton Pattern Implementation



# Factory Pattern (again)

- **For the purposes of this course, think of the Factory Pattern as simply moving construction logic to a separate “factory” class.**
- This is different to how the Factory Pattern is presented elsewhere.
- In the more typical definition of the Factory Pattern, factories are used to create objects *without specifying their concrete types*.

# Factory Pattern: Implementation



# Abstract Factory Pattern

- Sometimes, you may need to model objects where there are different **variations** or **categories** of those objects.
- For example, clothing items can be categorised by colour, size, brand etc.
- The Abstract Factory Pattern is intended for situations like this, where specific factories can be used to create objects of a particular category.

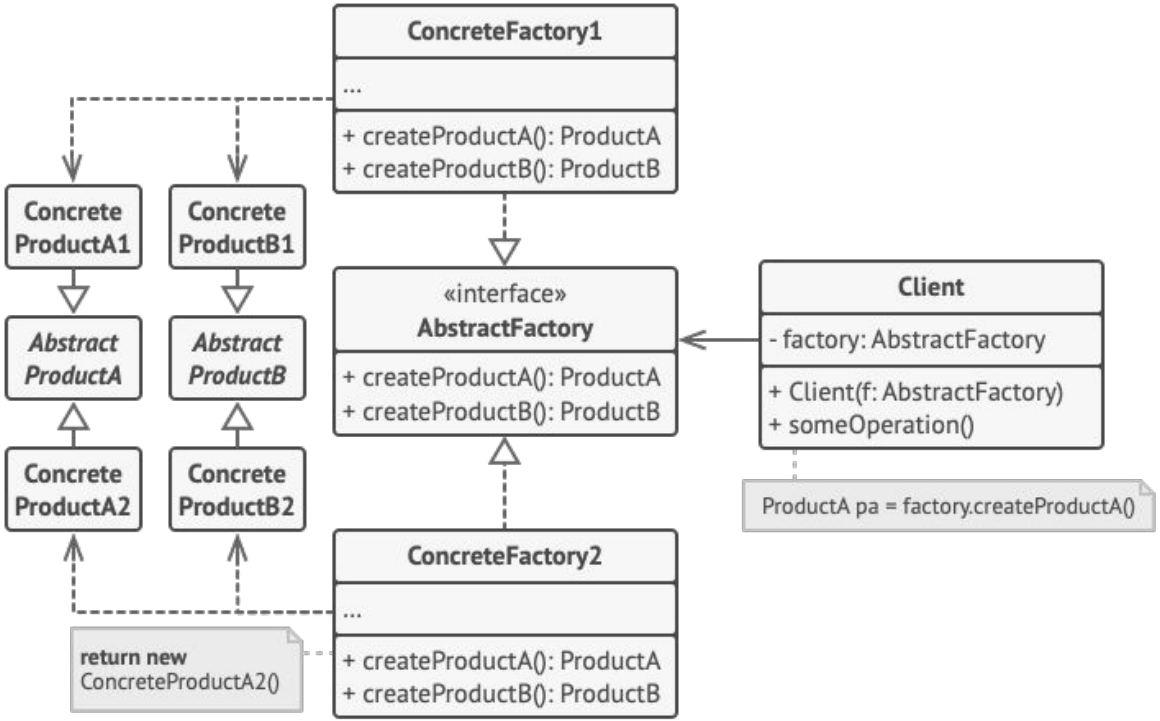
# Abstract Factory Pattern

- The Abstract Factory Pattern is a design pattern that provides interfaces to create **families of related objects** without specifying their concrete classes.
- The Abstract Factory Pattern defines a common interface defining the objects that each factory should be able to produce. Different factories for each category get made, implementing the interface.
    - This ensures that objects under the same category are kept together and makes changing between different categories simple (just change the factory being used).
    - Fundamentally, it is almost the same implementation as the Strategy Pattern.

# Abstract Factory Pattern

- For the earlier clothing example, the Abstract Factory Pattern implementation could look like:
  - A **`ClothingFactory`** interface with **`createShirt()`** and **`createPants()`** methods
  - Concrete classes such as **`SmallClothingFactory`**, **`MediumClothingFactory`**, **`LargeClothingFactory`** etc. which implement the **`ClothingFactory`** interface.
  - In the **`SmallClothingFactory`**, **`createShirt()`** returns a **`SmallShirt`**, **`createPants()`** returns a **`SmallPants`**.
  - In the **`MediumClothingFactory`**, **`createShirt`**( ) returns a **`MediumShirt`**, **`createPants`**( ) returns a **`MediumPants`** and so on.

# Abstract Factory Pattern: Implementation



# Live Example: Abstract Factory Pattern

- Inside **src/outfits**, there is a command-line system modelling a clothing store selling casual and formal outfits.
  - For casual outfits, the store recommends a T-shirt and jeans.
  - For formal outfits, the store recommends an Oxford shirt and trousers.
- Apply the Abstract Factory Pattern to improve the quality of the code.